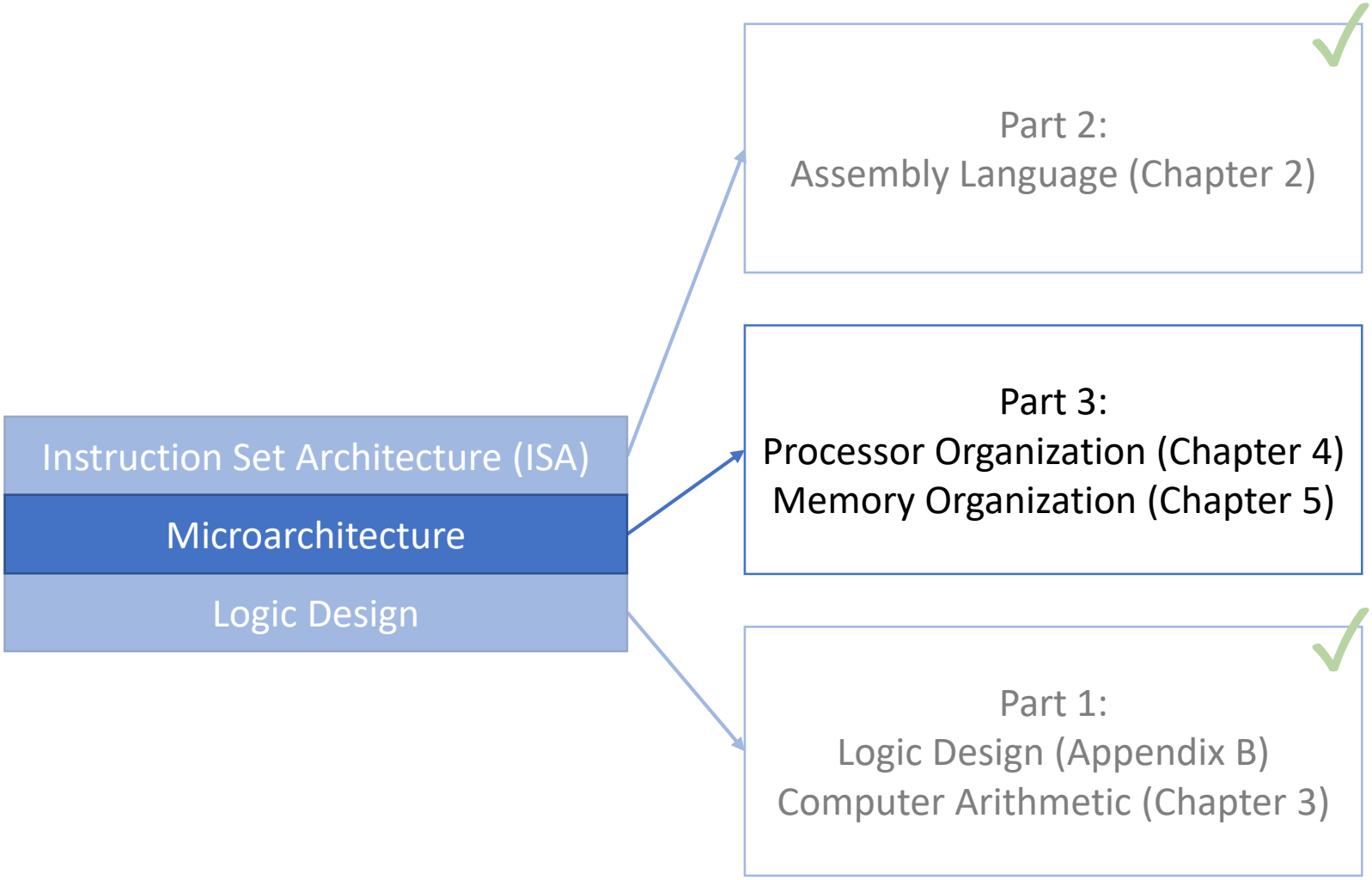
Lecture 29: Memory and Cache – Part 2

CMPS 221 – Computer Organization and Design

Adapted by Izzat El Hajj from textbook slides for Computer Organization and Design, by David Patterson and John Hennessy, Morgan Kaufmann.

Last Time: Memory and Cache



Memory Technology

Memory Technology	Access Latency	$/GiB in 2012
SRAM	0.5-2.5 ns	$500-$1,000
DRAM	50-70ns	$10-$20
Flash	5,000-50,000 ns	$0.75-$1.00
Magnetic Disk	5,000,000-20,000,000 ns	$0.05-$0.10

- Ideal memory:
  - Access latency of SRAM
  - Capacity and cost/GiB of disk
- Strategy: arrange memory in a *hierarchy*
  - Smaller and faster memory for data currently being accessed
  - Larger and slower memory for data not currently being accessed

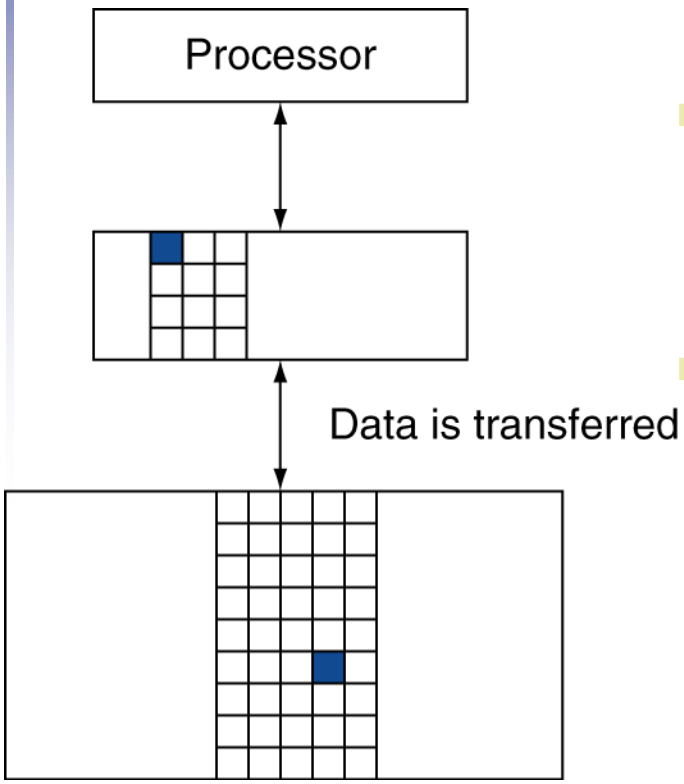
Memory Hierarchy

- Arrange memory in a **hierarchy**
- Store all data on a large flash or magnetic **disk**
- Copy recently accessed (and nearby) items from disk to a smaller DRAM memory
  - Called **main memory**
- Copy more recently accessed (and nearby) items from DRAM to a smaller SRAM
  - Called **cache memory**, closest to the CPU
- “Recently” and “nearby” are good predictors of “currently” because of the *principle of locality*

Principle of Locality

- The **principle of locality** is the observation that programs access a small proportion of their address space at any time
- **Temporal locality**
  - Items accessed recently are likely to be accessed again soon (i.e., locality across time)
  - e.g., instructions in a loop, induction variables
- **Spatial locality**
  - Items nearby those accessed recently are likely to be accessed soon (i.e., locality across space)
  - e.g., sequential instruction access, array data

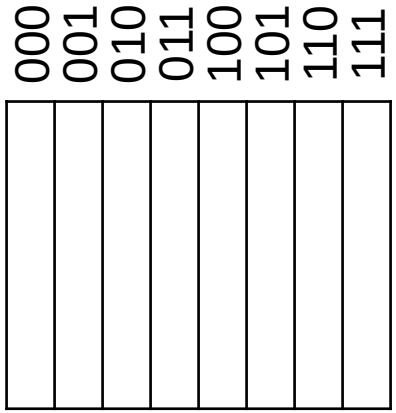
Memory Hierarchy Levels



- A **block** (or **line**) is a unit of copying
  - May be multiple words
- A **hit** at a level occurs if accessed data is present at the level
  - **Hit ratio**: hits/accesses
- A **miss** at a level occurs if accessed data is not present at the level and needs to be fetched from the next
  - **Miss penalty**: time to fetch data from the next level
  - **Miss ratio**: misses/accesses
    - miss ratio = 1 – hit ratio
  - Then accessed data is stored in and supplied from the closest level

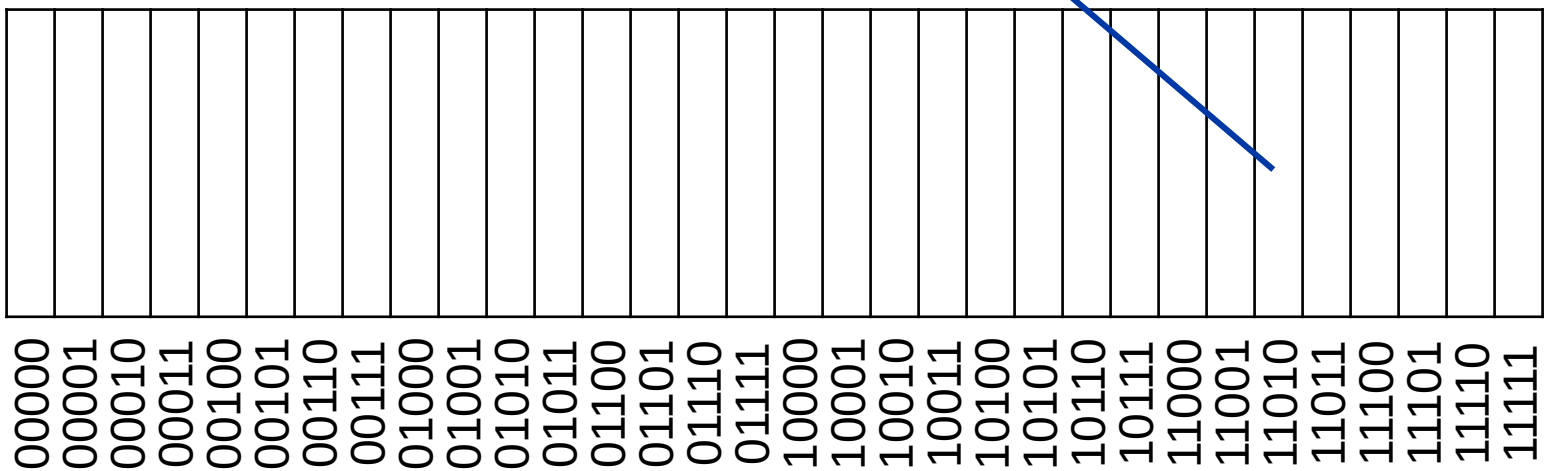
Block Placement

Consider a cache with (2^3) 8 blocks indexed with 3 bits



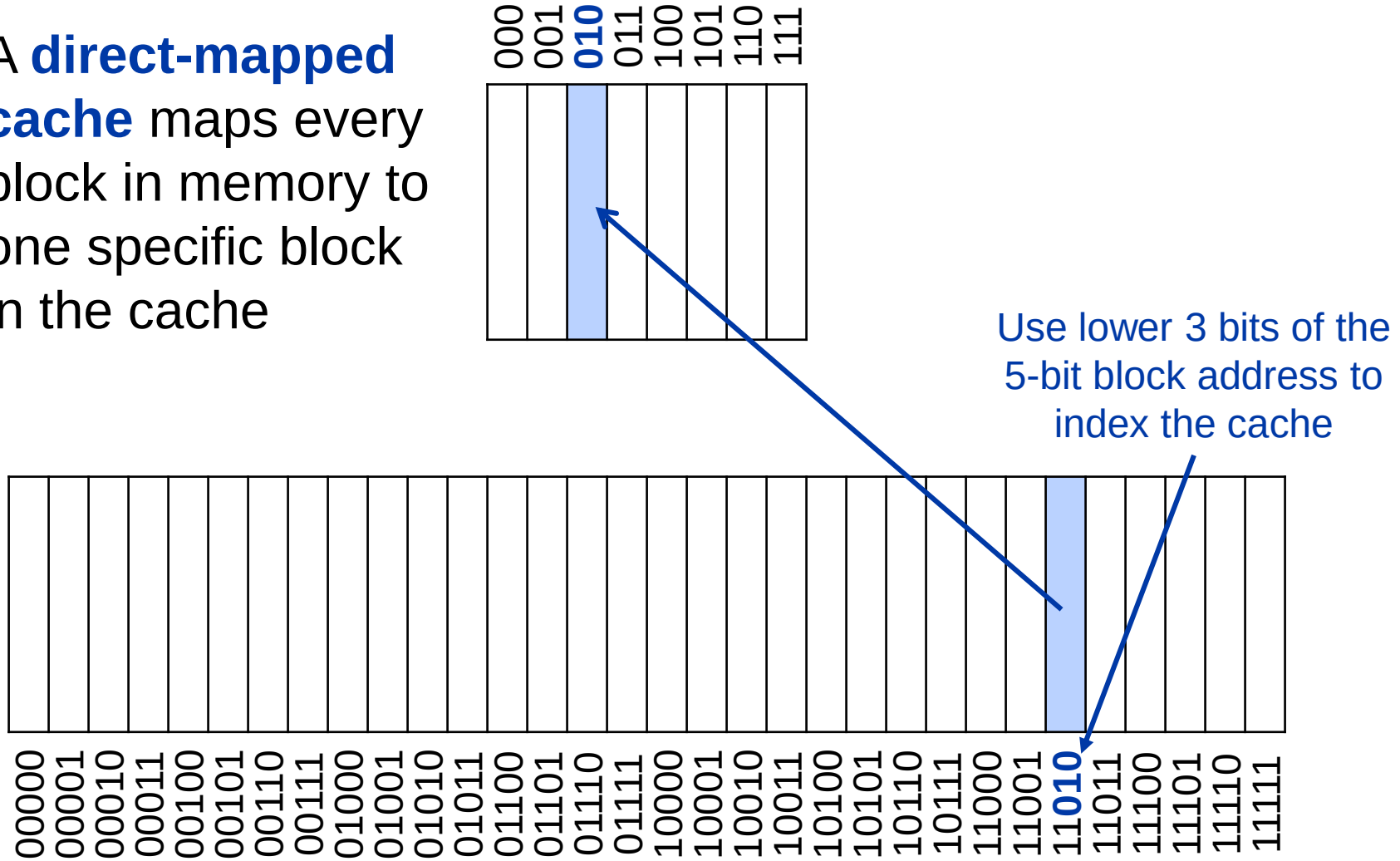
Consider a memory with 2^5 (32) blocks and 5-bit block addresses

When copying a block from memory to cache, where do we place it?



Direct Mapped Cache

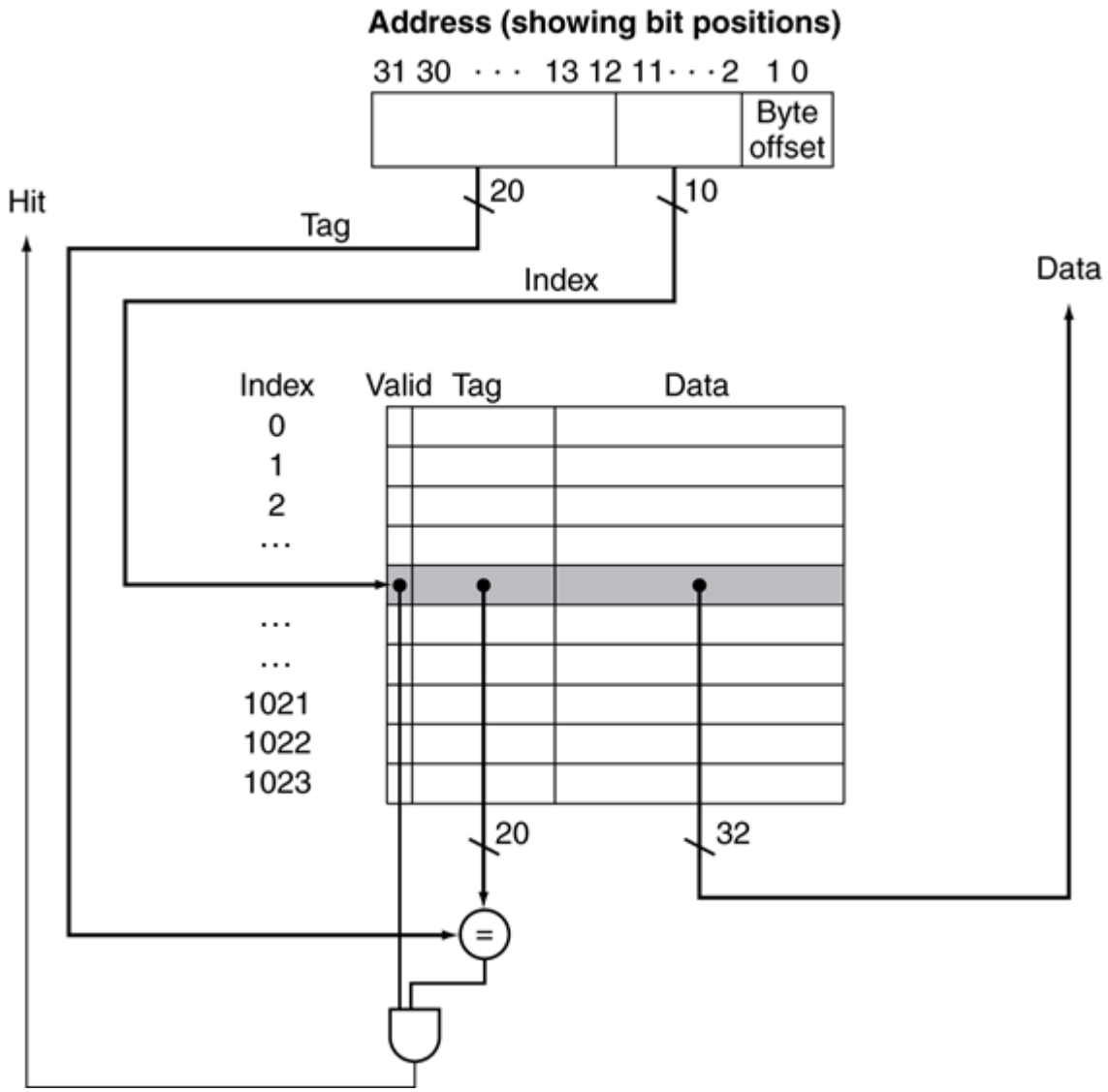
A **direct-mapped cache** maps every block in memory to one specific block in the cache



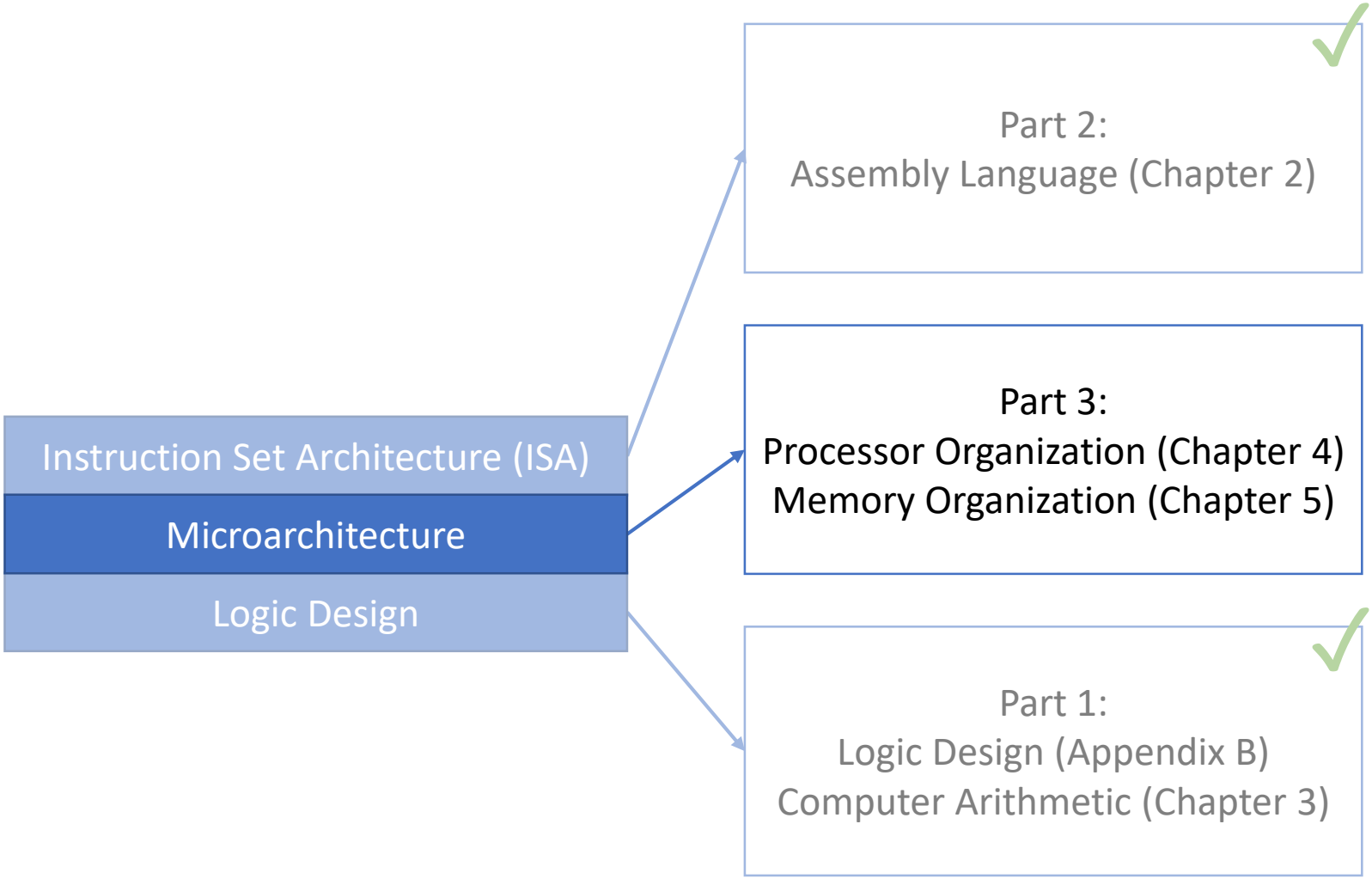
Tags and Valid Bits

- How do we know which memory block is stored in a cache location?
  - Store the address of the block with the data
    - Only higher bits are needed (index is implicit)
    - Called the **tag**
- What if there is no data in a location?
  - **Valid bit**: 1 = present, 0 = not present
  - Initially 0

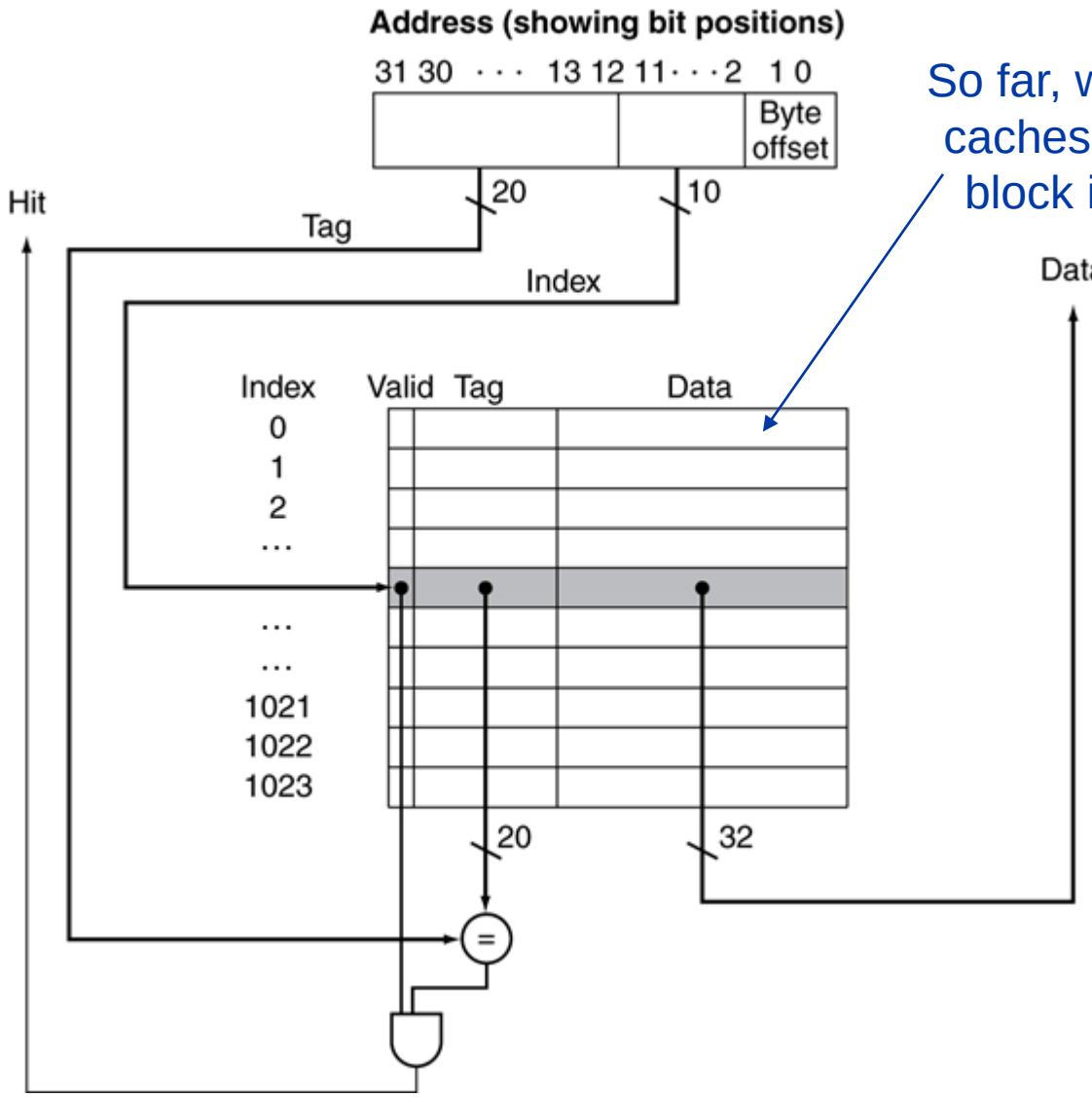
Address Subdivision



Today: Memory and Cache



Recall: One-word Blocks

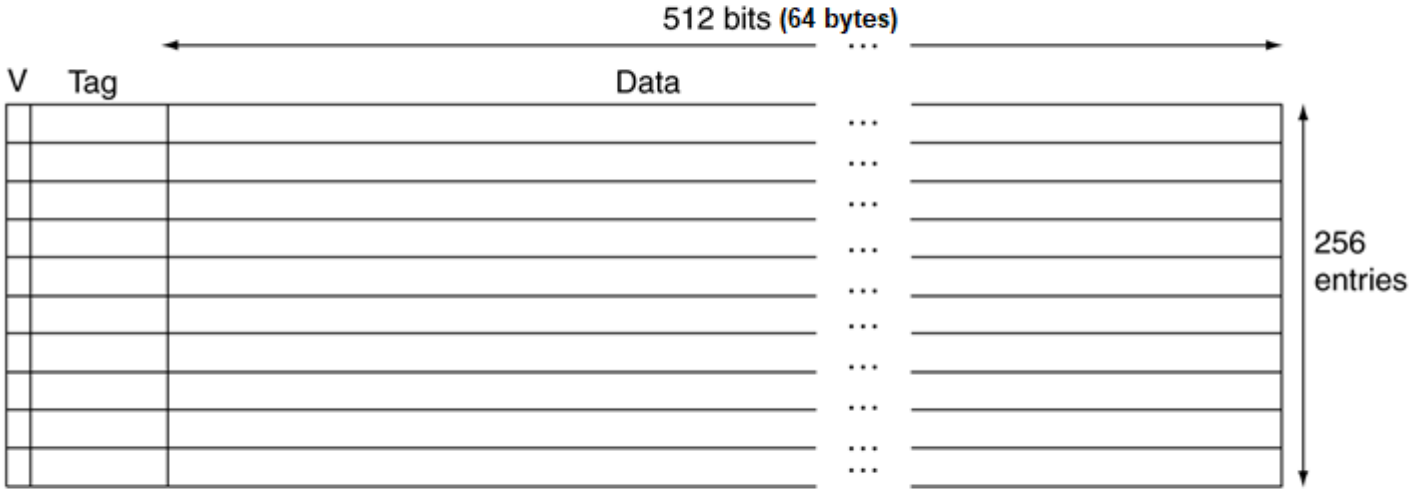


So far, we have seen caches where each block is one word

What if we make blocks larger than one word?

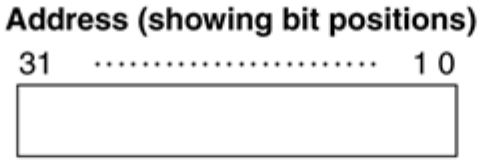
Caches with Large Block Size

Consider a cache with 2^8 (256) blocks and 64 bytes/block

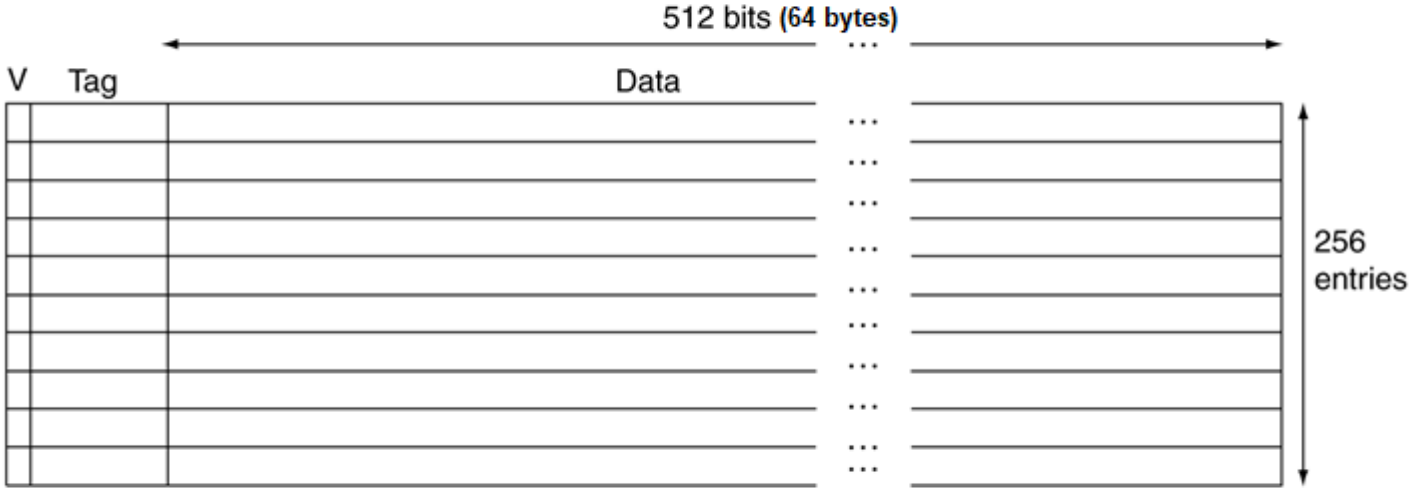


Caches with Large Block Size

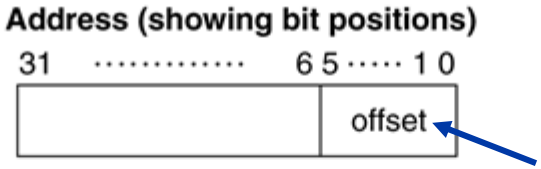
Consider a cache with 2^8 (256) blocks and 64 bytes/block



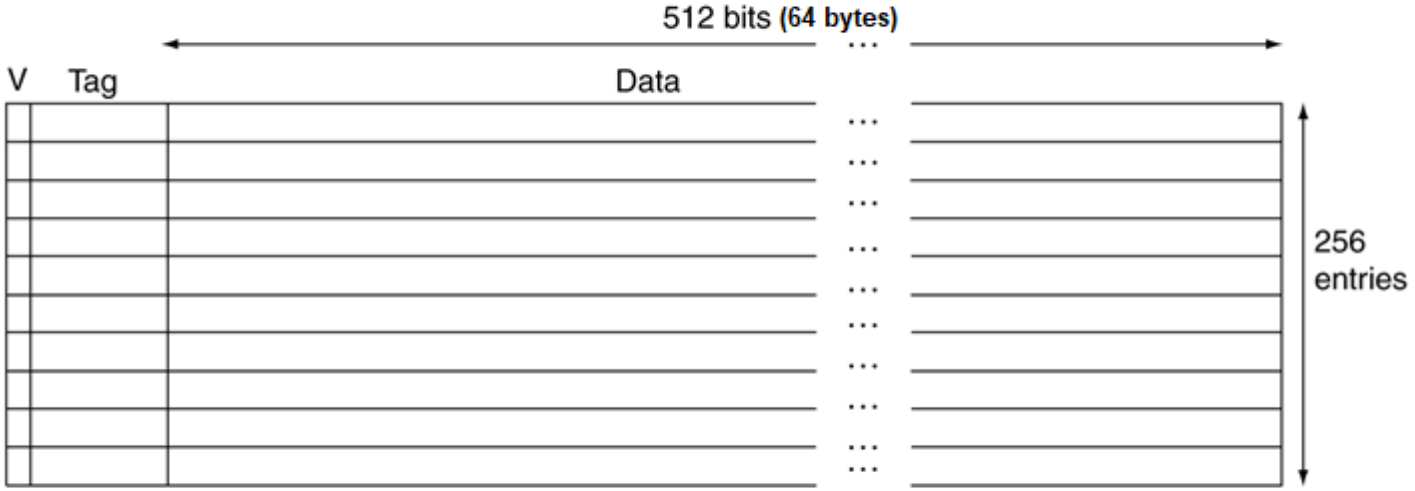
How do we subdivide a 32-bit memory address to access the cache?



Caches with Large Block Size

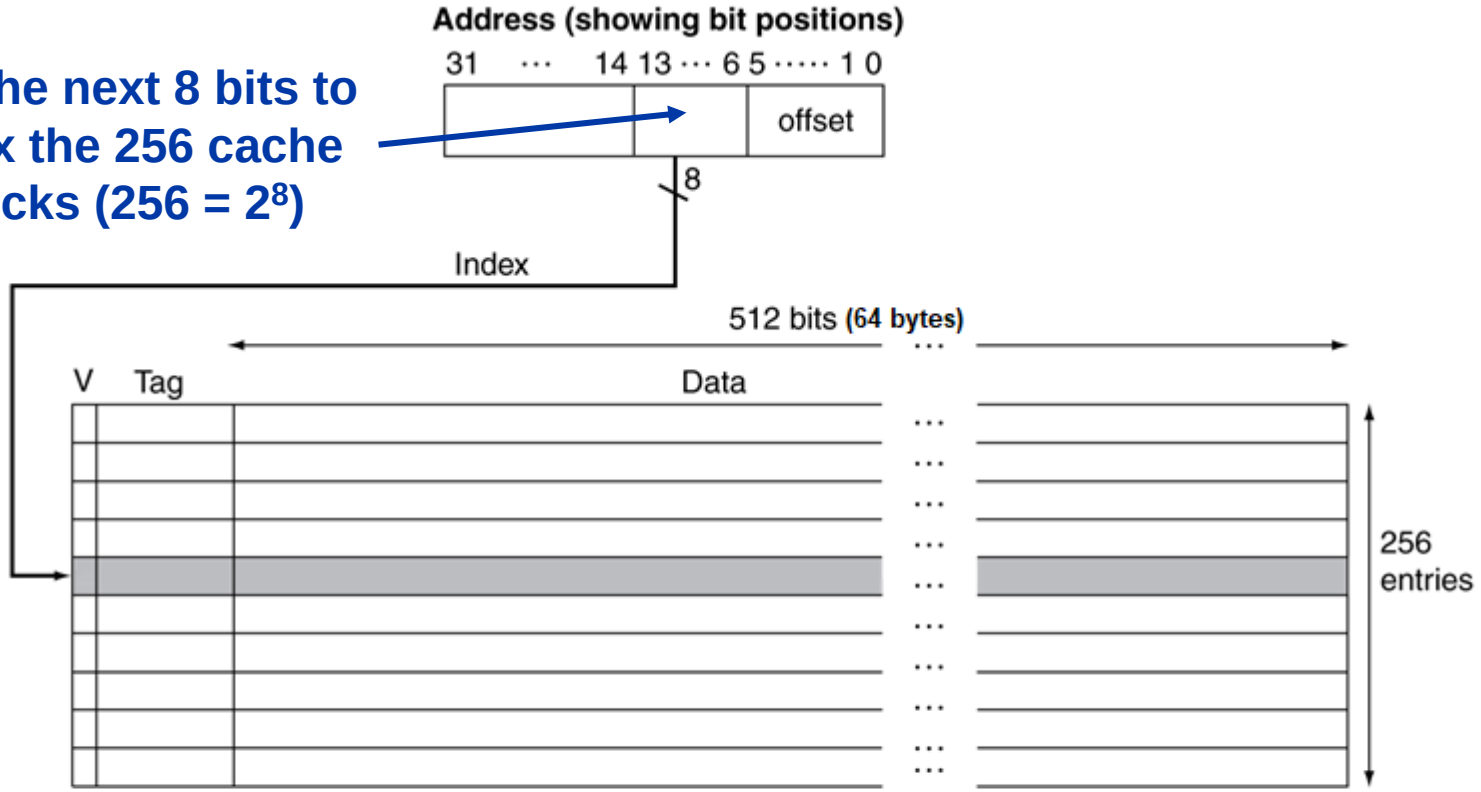


Lowest 6 address bits differentiate bytes in the same block (64 = 2^6)

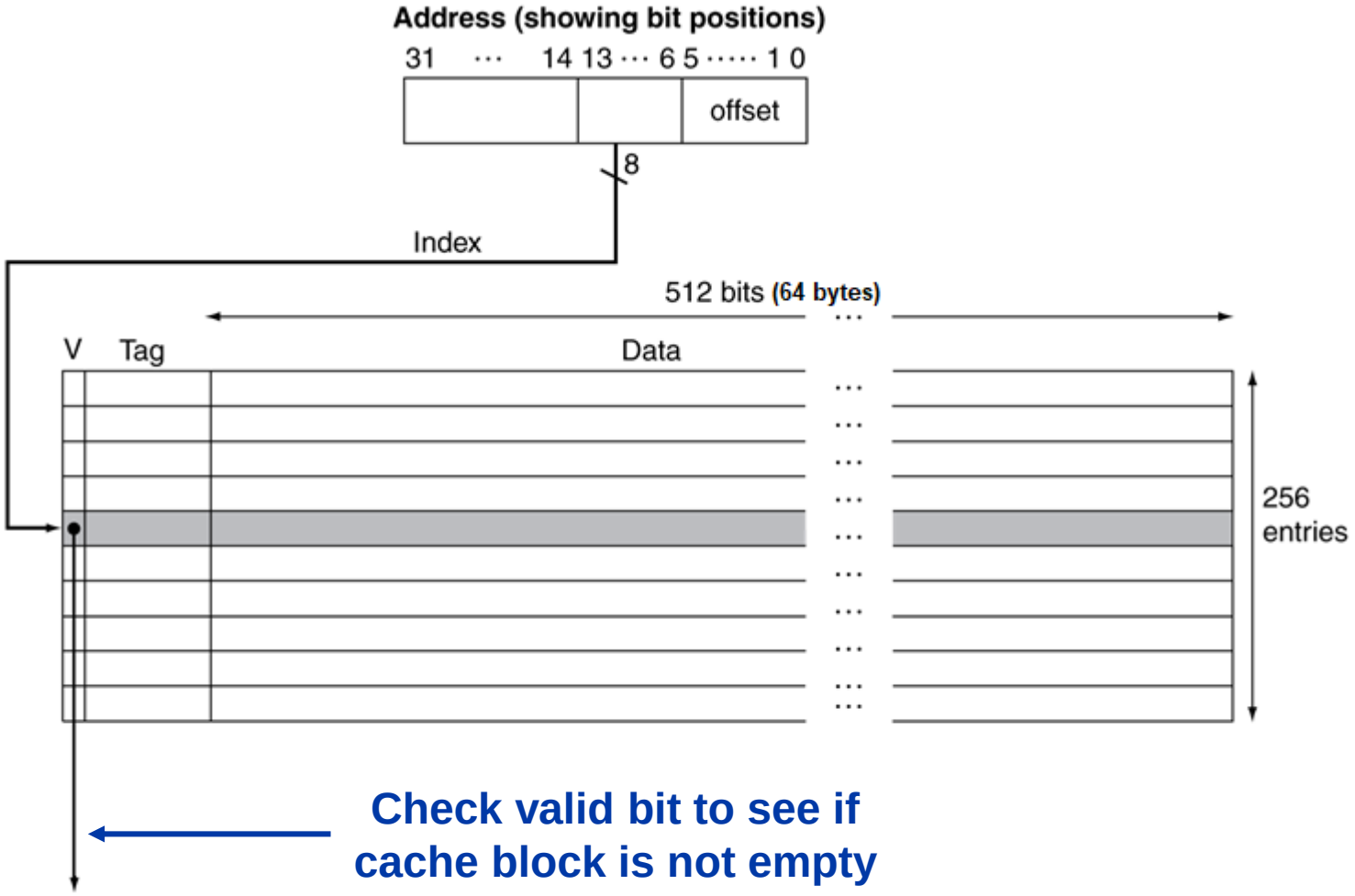


Caches with Large Block Size

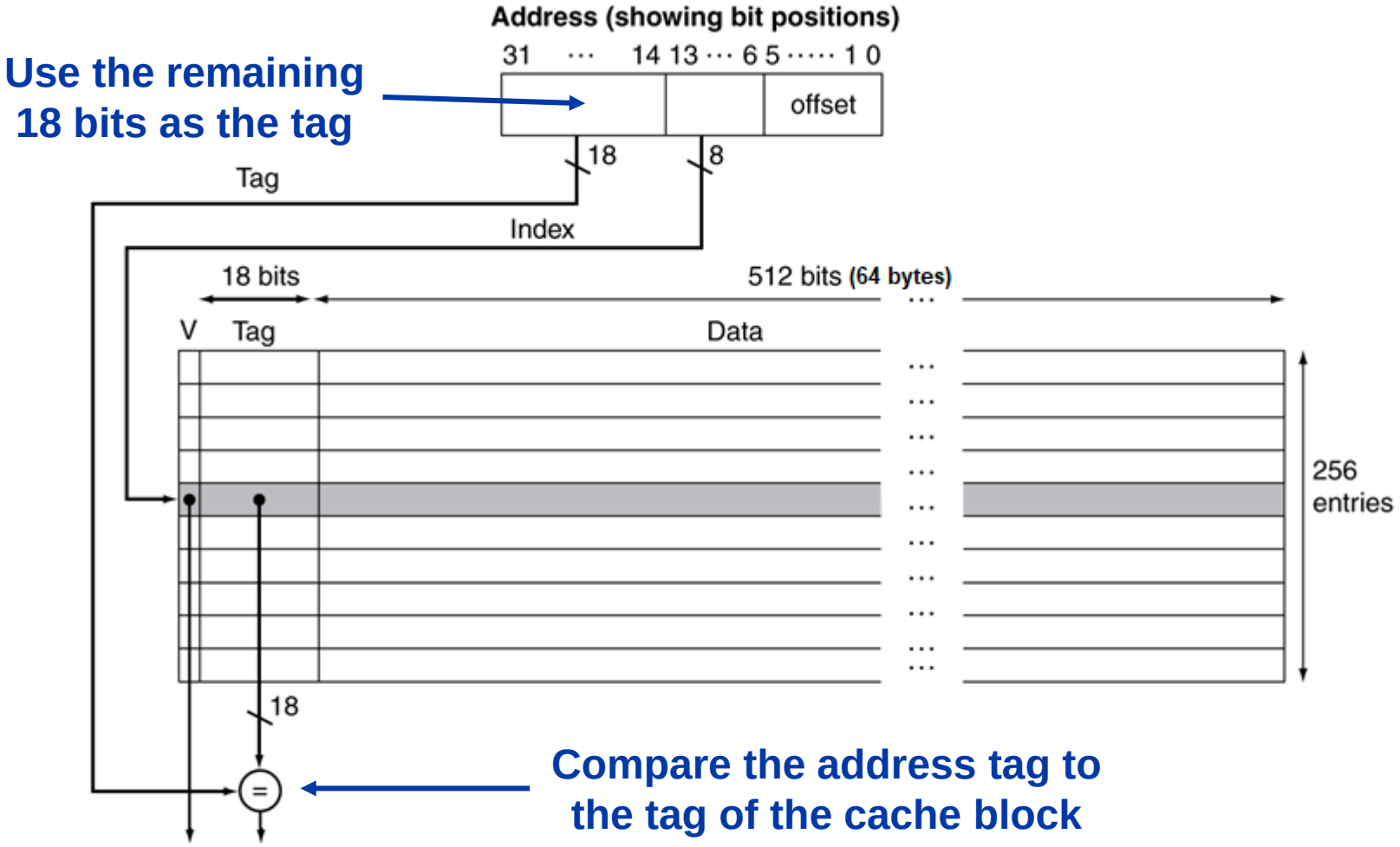
Use the next 8 bits to index the 256 cache blocks (256 = 2^8)



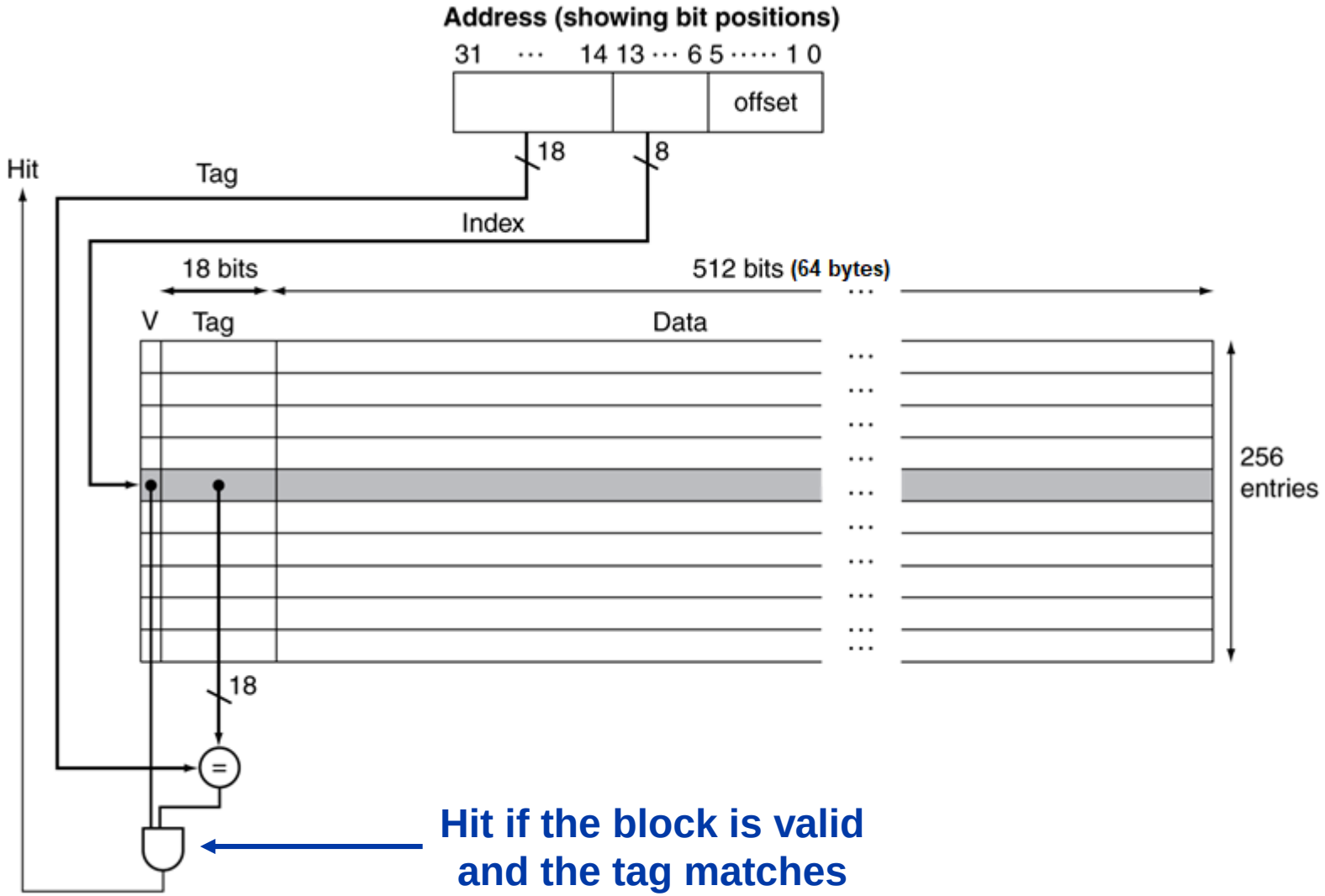
Caches with Large Block Size



Caches with Large Block Size

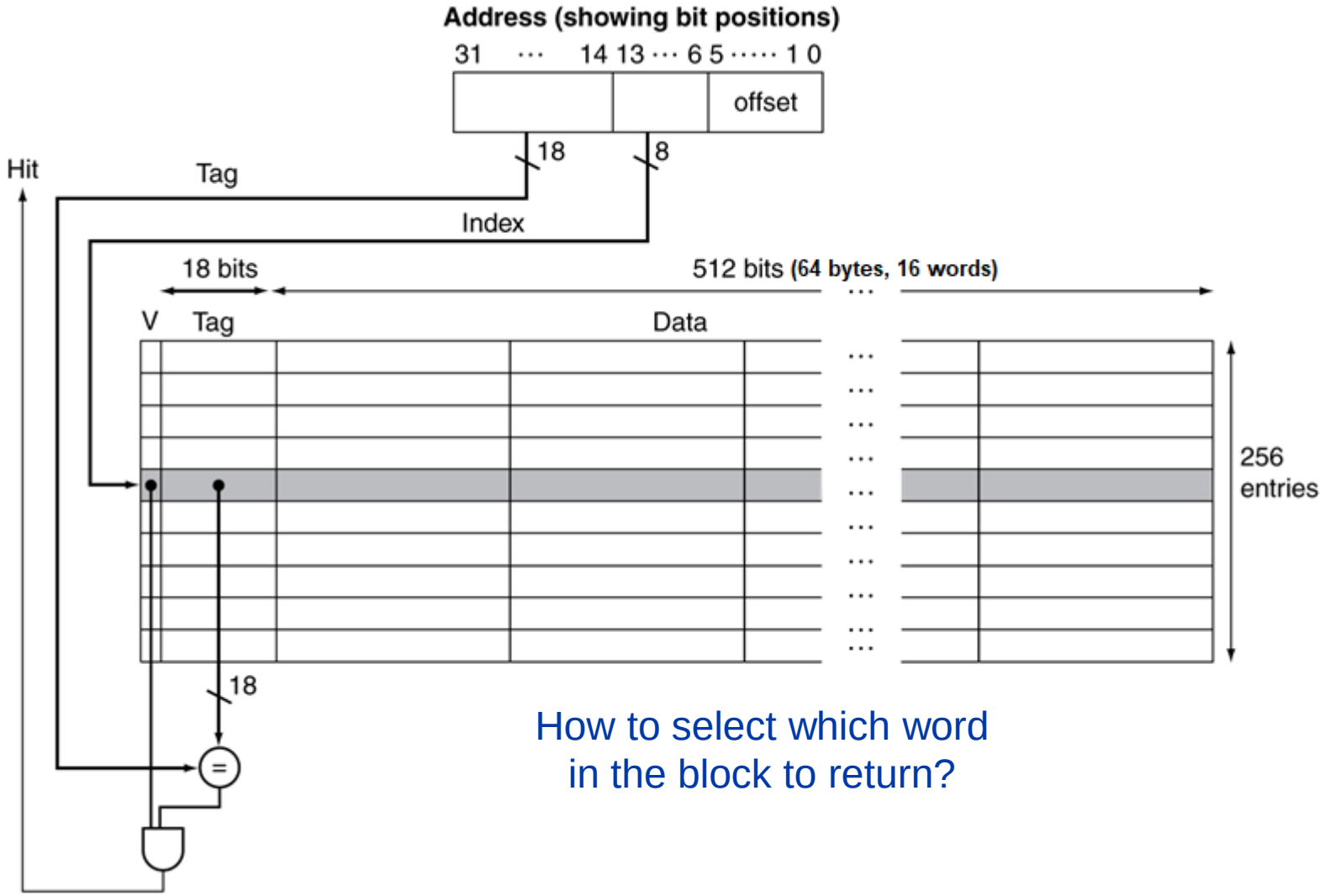


Caches with Large Block Size



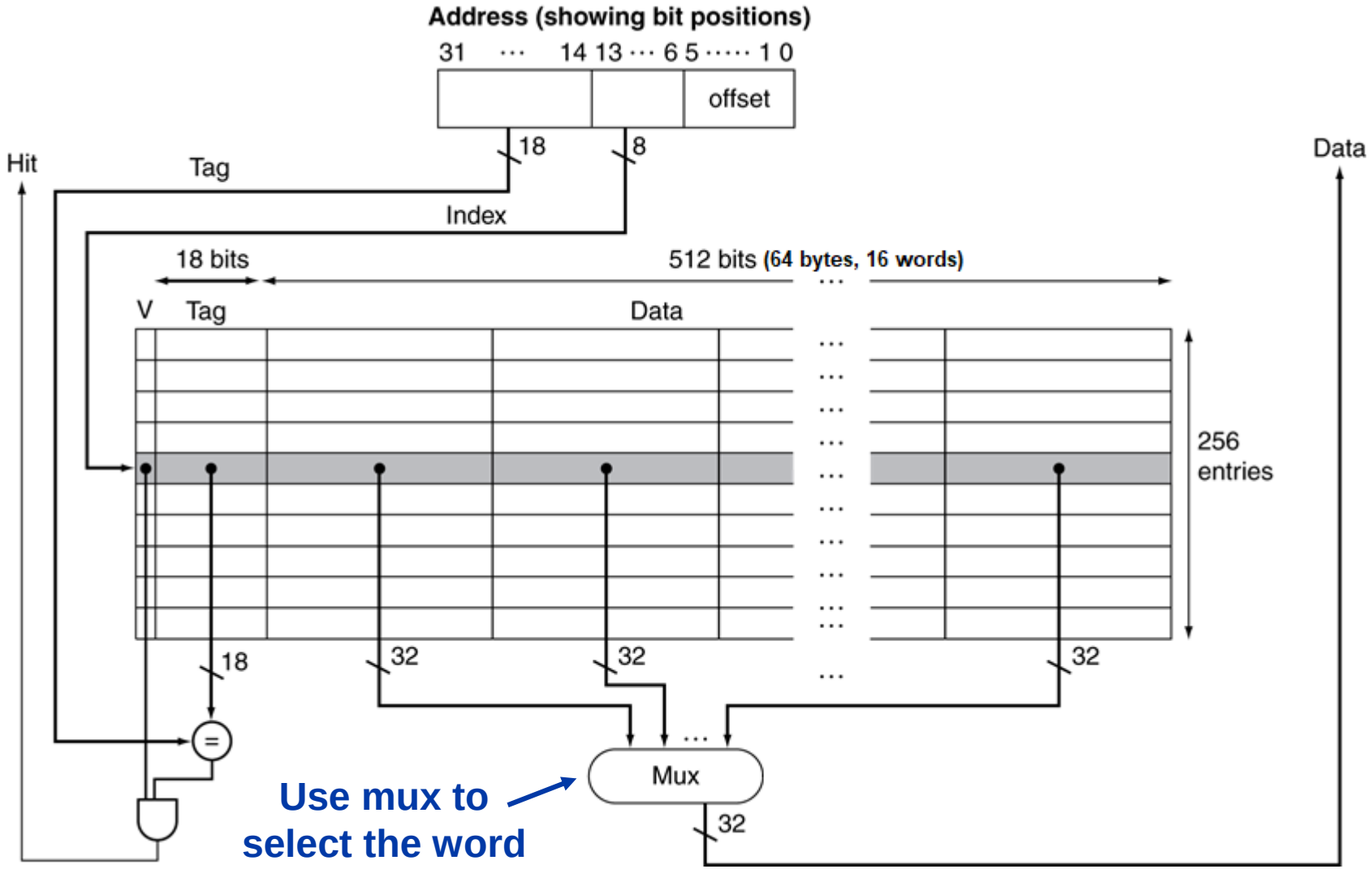
Hit if the block is valid and the tag matches

Caches with Large Block Size

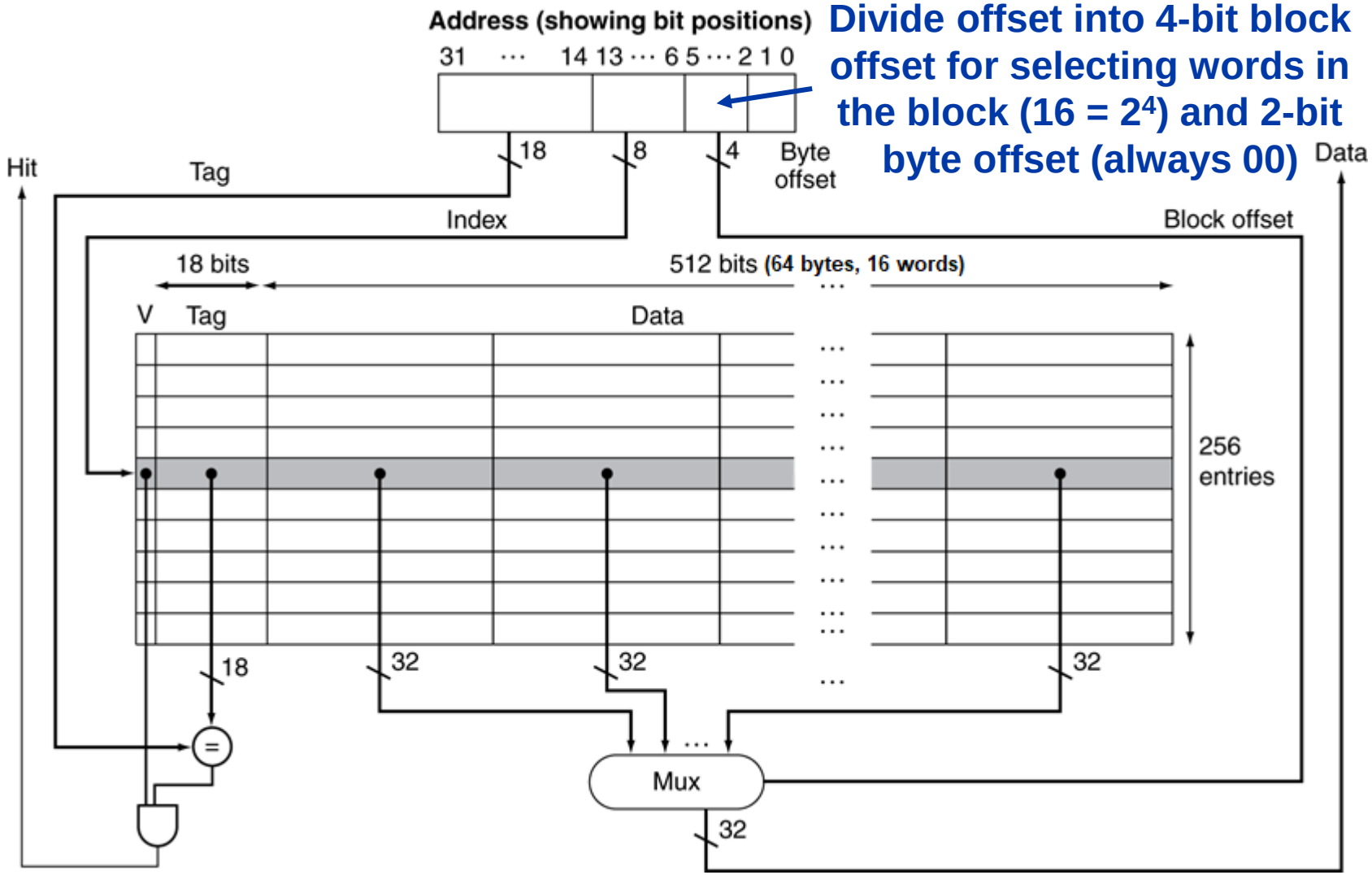


How to select which word in the block to return?

Caches with Large Block Size



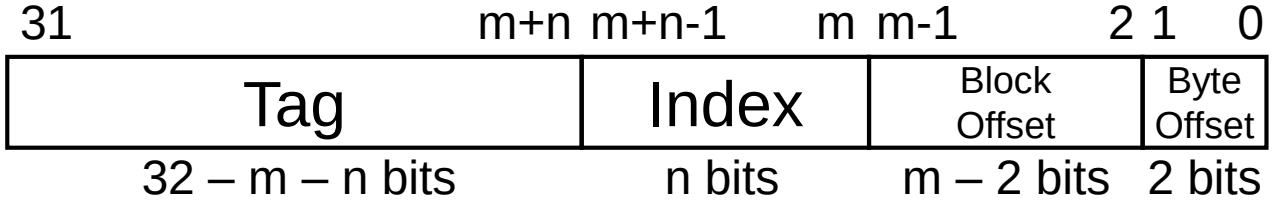
Caches with Large Block Size



Divide offset into 4-bit block offset for selecting words in the block ($16 = 2^4$) and 2-bit byte offset (always 00)

Direct Mapped Cache Address Bits

- Cache with 2^n blocks, 2^m bytes/block

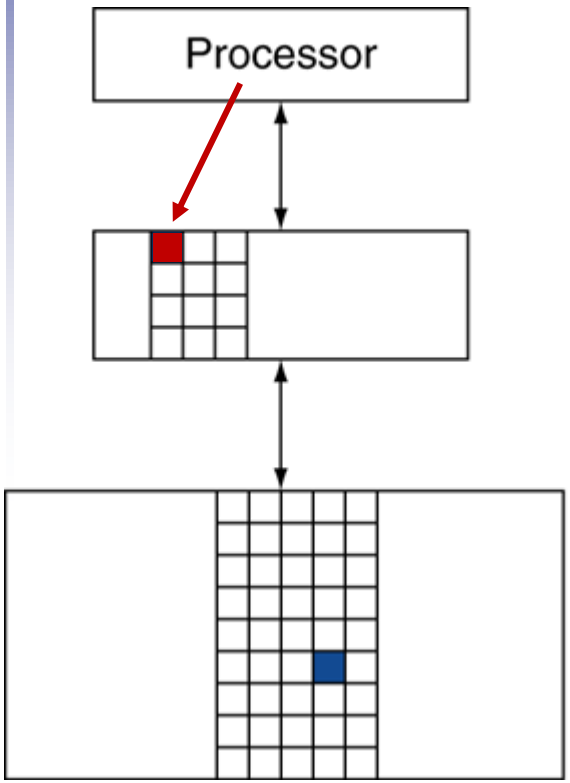


- Size of a direct mapped cache
  - Data bits = $2^n \cdot 2^m \cdot 8$
  - Valid bits: 2^n (1 bit per block)
  - Tag bits: $2^n \cdot (32 - m - n)$ (1 tag per block)
  - Total bits = $2^n \cdot (1 + [32 - m - n] + 2^m \cdot 8)$
  - Efficiency: $\frac{data\ bits}{total\ bits} = \frac{2^m \cdot 8}{33 - m - n + 2^m \cdot 8}$

Impact of Block Size

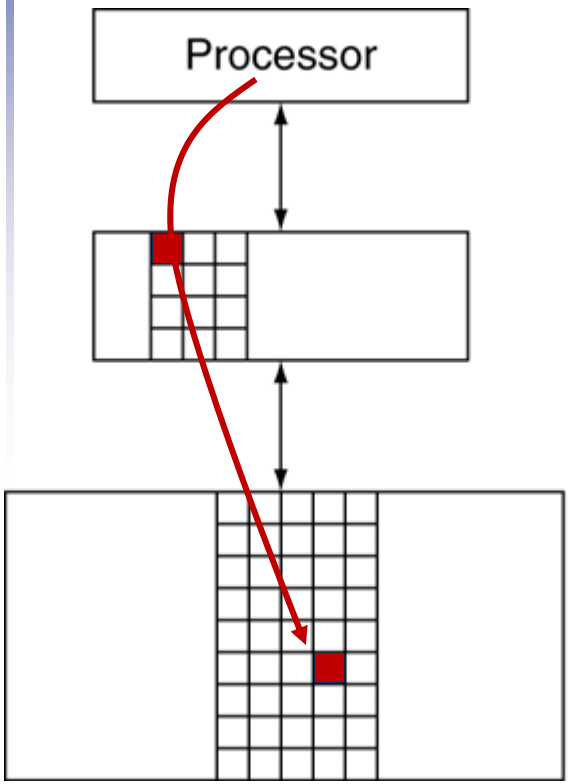
- Advantages of larger blocks:
  - Amortize the overhead of the tag bits
  - Reduces miss rate due to spatial locality
- Disadvantages (assuming fixed-size cache):
  - Fewer blocks (more conflicts which increases miss rate)
  - Underutilizes blocks (pollution) if no spatial locality
  - Larger miss penalty (more bytes to fetch)
    - Can override benefit of reduced miss rate
    - Early restart and critical-word-first can help

Writing to Caches



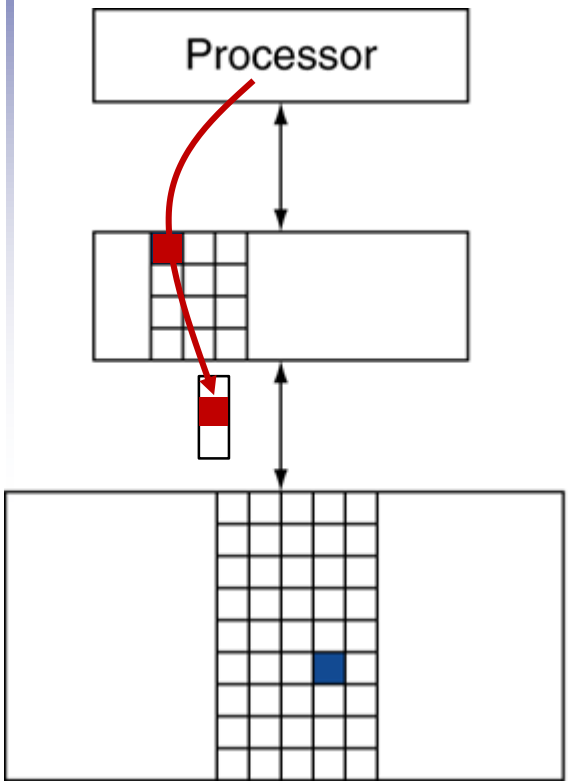
- On a **write hit**, if we just update the block in cache, cache and memory would be inconsistent
- Need to ensure that both are eventually updated

Write-Through Cache



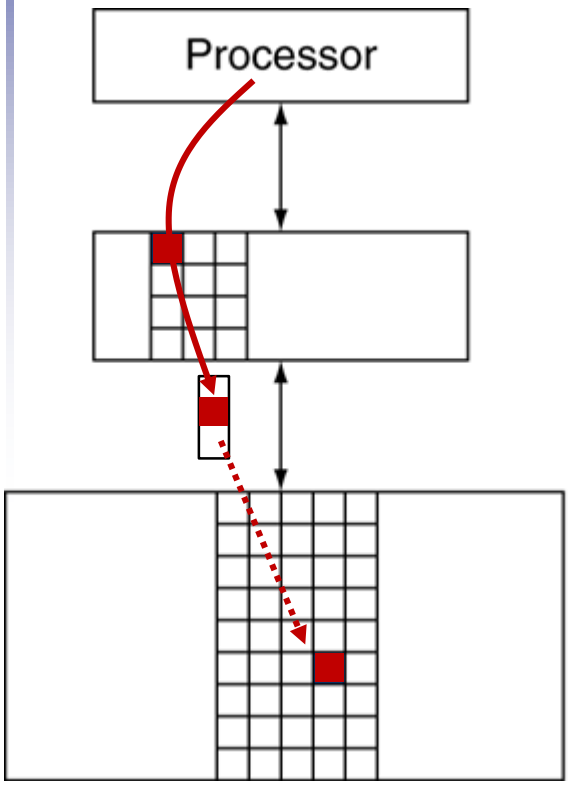
- A **write-through cache** updates both cache and memory at the time of the write
- Disadvantage
  - Makes writes take longer because they need to wait for next level in the hierarchy

Write-Through Cache



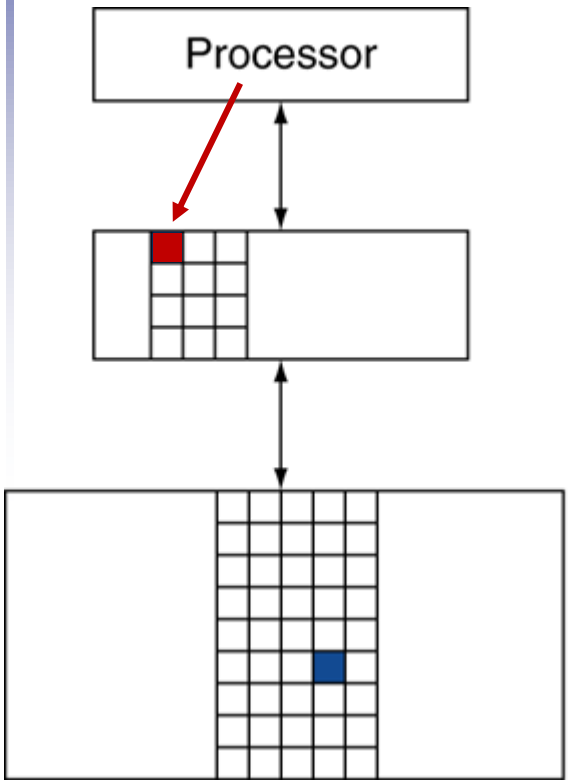
- A **write-through cache** updates both cache and memory at the time of the write
- Disadvantage
  - Makes writes take longer because they need to wait for next level in the hierarchy
- Solution: use a **write buffer**
  - Buffers data that is waiting to be written to memory

Write-Through Cache



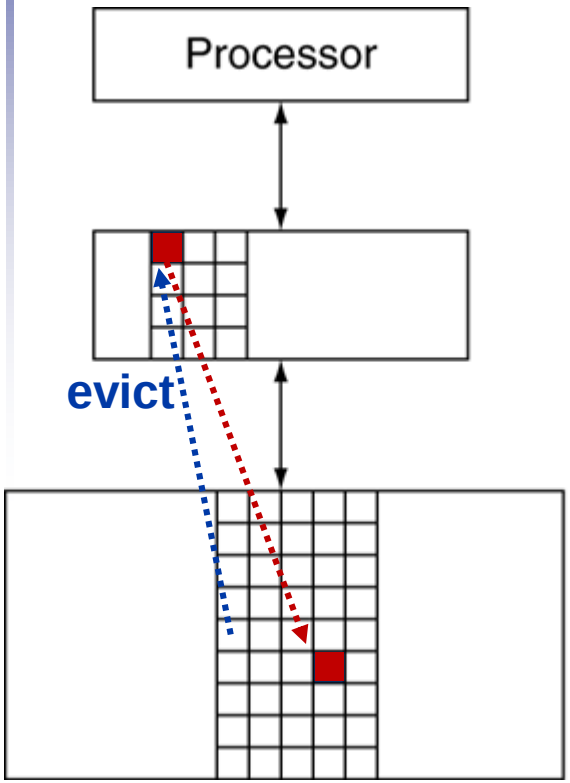
- A **write-through cache** updates both cache and memory at the time of the write
- Disadvantage
  - Makes writes take longer because they need to wait for next level in the hierarchy
- Solution: use a **write buffer**
  - Buffers data that is waiting to be written to memory
  - Processor continues while write buffer writes in the background
    - Only stalls if write buffer is already full

Write-Back Cache



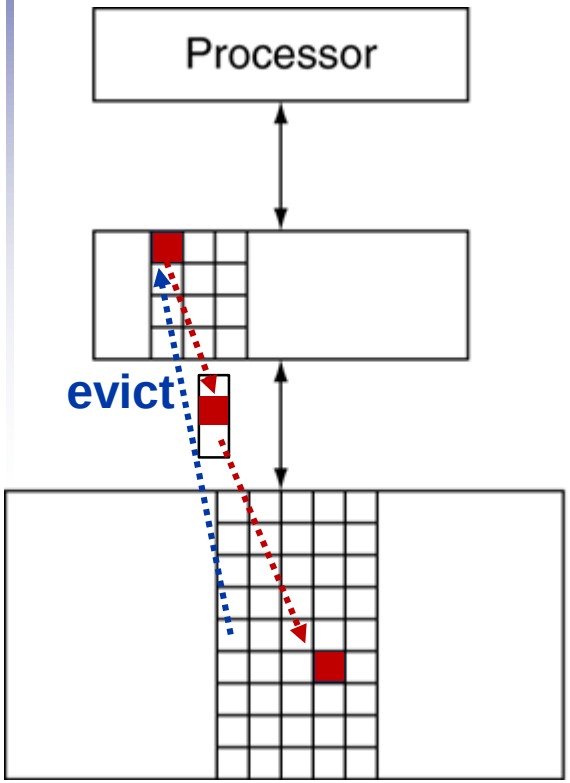
- A **write-back cache** only updates the cache

Write-Back Cache



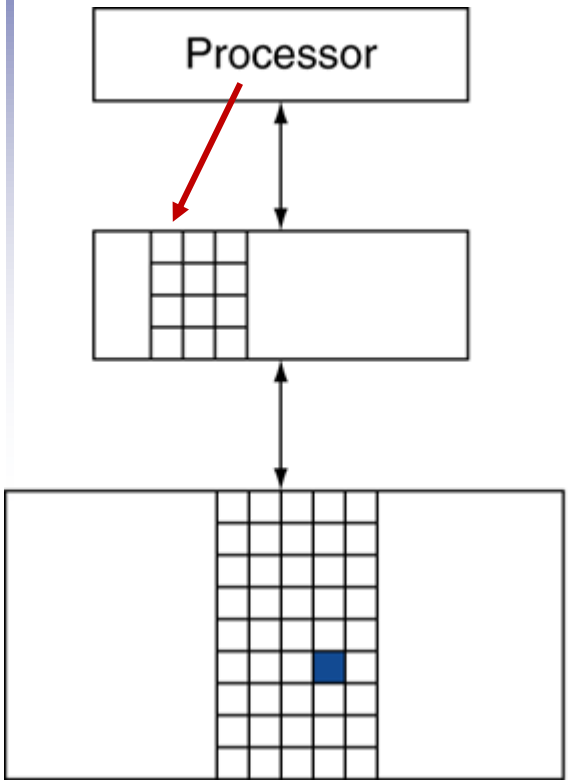
- A **write-back cache** only updates the cache
- Value is written to memory when the block is evicted
  - Need to know which blocks have changed
  - Use a **dirty bit**

Write-Back Cache



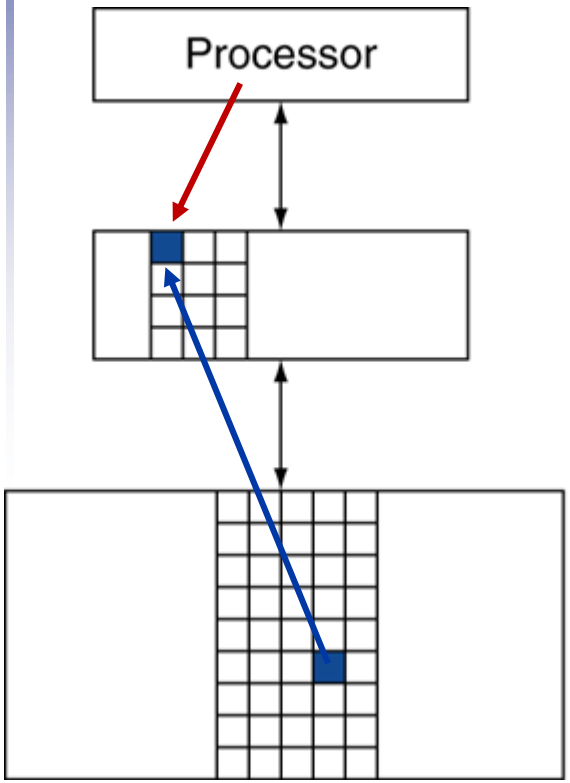
- A **write-back cache** only updates the cache
- Value is written to memory when the block is evicted
  - Need to know which blocks have changed
  - Use a **dirty bit**
- Now evictions take long
  - Solution: use a **write buffer** for evicted dirty blocks

Writing to Caches



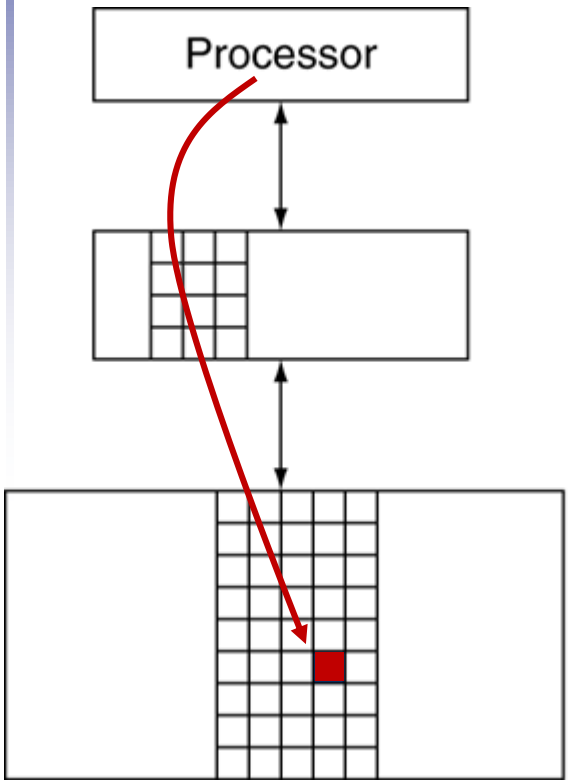
- On a **write miss** we can fetch the block or just update it in memory

Write-Allocate Cache



- A **write-allocate cache** fetches the block to the cache before writing it
  - Works with both write-through and write-back caches

Write-Around Cache



- A **write-around cache** writes directly to memory without fetching the block
  - Only works for write-through caches
  - Useful since programs often write a whole block before reading it (e.g., initialization)

Writing to Caches Summary

- On a **write hit**:
  - A **write-through cache** updates memory
  - A **write-back cache** only updates the block in the cache (and sets a **dirty bit**)
  - Use a **write buffer** to avoid waiting for write-throughs or write-backs to complete
- On a **write miss**:
  - **Write-allocate**: fetch the block
  - **Write-around**: do not fetch the block

Textbook Sections

- The content in these slides corresponds to:
  - Textbook:
    - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
  - Sections:
    - 5.3